

Technical Notes

NOTES/DOCUMENTATION

- LangChain:
 - Agents: [🔗 Workflows and agents - Docs by LangChain](#) [🔗 Agents - Docs by LangChain](#)
 - Model: select LLM for agent
 - **ToolNode**: can define tools w/ @ tools decorator and bind them to LLM
 - receive arguments generated by the model as their first argument
 - basic node structure:
 - **LLM call node**: LLM decides whether to call the tool or not, and always returns an output message about what the tool performed
 - **tool node**: actually performs the tool
 - **conditional edge**: conditional edge function that routes the tool node, or ends it, based on if LLM made a tool call
 - in essence, continue the loop or stop?
 - if LLM makes a tool call, run the tool node
 - else, end and reply to user
 - System prompt
 - accepts a string, ex. "You are a helpful assistant. Be concise and accurate."
 - agent:
 - build agent by adding these nodes
 - connect these nodes with edges or conditional edges
 - Regular edge: START ↔ LLM call
 - Conditional edge: LLM call ↔ conditional edge ↔ tool node
 - Regular edge: tool_node ↔ LLM call (if in loop)
 - Structured output:
 - Invoke:
 - Agent's State is updated
 - Returns final response at the end of the run
 - If multiple tool calls are executed, the user can track progress with Streaming
 -
 - **Planning/Delegation**
 - Used for complex tasks > 1 context window
 - Main agent breaks work into pieces to run in parallel

- **Guardrails:**
 - enforce deterministically regardless of model behavior
 - intercept data as it flows through agent loop
 - apply rules before tool results reach model's context
 - Middleware examples: [🔗 Prebuilt middleware - Docs by LangChain](#)
 - possibly use:
 - **Model call limit**
 - Tool call limit
 - PII detection
 - Human in the loop: steering, human decides before expensive API calls
 - Custom Middleware: [🔗 Custom middleware - Docs by LangChain](#)
 - Node-style (runs at specific execution points)
 - @ before_agent | after_agent
 - @ before_model | after_model
 - Wrap-style (run around each model)
 - @ wrap_model_call
 - @ wrap_tool_call
 - Class-based middleware
 - for complex middleware with multiple hooks or configuration
 - when you need to define both sync and async implementations for the same hook, or want to combine multiple hooks in a single middleware
 - complex config required (custom models, configurable thresholds)
 - reuse across projects with init-time config
 - Execution flow
 - Before hooks run in order (1 → 2 → 3)
 - After hooks run in reverse (3 → 2 → 1)
 - Wrap hooks: nested, 1 is outermost, 3 is innermost
- OpenAI Agents SDK:
 - Agents: [🔗 Agents - OpenAI Agents SDK](#)
 - Memory: [🔗 Agent memory - OpenAI Agents SDK](#)
 - Human in the loop: [🔗 Human-in-the-loop - OpenAI Agents SDK](#)
 - Usage (Cost/Tokens per run): [🔗 Usage - OpenAI Agents SDK](#)
 - MCP: [🔗 Model context protocol \(MCP\) - OpenAI Agents SDK](#)

- Anthropic Agent SDK:

- Agent loop: [🔥 How the agent loop works - Claude Code Docs](#)
 - Receive prompt → Evaluate current state and respond (with text/1+ tool calls) → HOOK Execute Tools (and collect result) → Repeat (eval and execute) → Return result
 - Set budget for turns, especially for long, open-ended prompts
- Custom Tools: [🔥 Give Claude custom tools - Claude Code Docs](#)
- External Tools w MCP: [🔥 Connect to external tools with MCP - Claude Code Docs](#)
- Tool Restrictions: [🔥 Agent Skills in the SDK - Claude Code Docs](#)
- Permissions: [🔥 Configure permissions - Claude Code Docs](#)
- Control agent behavior with hooks**: [🔥 Intercept and control agent behavior with hooks - Claude Code Docs](#)
 - Filesystem (settings.json)
 - Sharing hooks between CLI and SDK sessions. Supports "command" (shell scripts), "http" (POST to an endpoint), "_tool" (call a connected MCP server's tool), "prompt" (LLM evaluates a prompt), and "agent" (spawns a verifier agent). These fire in the main agent and any subagents it spawns
 - Programmatic (callbacks in query())
 - Application-specific logic, structured decisions, and in-process integration. These also fire inside subagents. The callback receives agent_id and agent_type to distinguish.
- Track usage: [🔥 Track cost and usage - Claude Code Docs](#)

LangGraph - Node/Edge Structure [🔗 Workflows and agents - Docs by LangChain](#)

-

LangChain Middleware - Safety Gate/Hooks [🔗 Prebuilt middleware - Docs by LangChain](#)

- —

Model Context Protocol - Tools [🔗 What is the Model Context Protocol \(MCP\)? - Model Context Protocol](#)

- MCP tools follow the naming pattern `mcp__<server-name>__<tool-name>`. For example, a GitHub server named "github" with a `list_issues` tool becomes `mcp__github__list_issues`.

[🔗 GitHub - modelcontextprotocol/servers: Model Context Protocol Servers](#)

Building own MCP server for read-only tools: [🔗 Build an MCP server - Model Context Protocol](#)

- retrieve relevant docs

- validate LLM-generated SQL
- cost estimation
- execute approved queries
- this lies BEFORE execution tools

SQL Validation (AST Validation) [sqlglot API documentation](#)

•

MVP Functions:

search_docs(query: str, scope: PermissionScope, top_k: int) → RetrievedContext
get_table_schema(tables: list[TableRef], scope: PermissionScope) → SchemaBundle
validate_plan(plan: QueryPlan, scope: PermissionScope) → GateDecision
generate_sql(plan: ApprovedQueryPlan) → SQLCandidate
check_sql(sql: str, plan: ApprovedQueryPlan, scope: PermissionScope) → GateDecision
dry_run_sql(sql: str, params: dict, scope: PermissionScope) → DryRunReport
approve_execution(sql_hash: str, plan_hash: str, dry_run: DryRunReport) → ApprovalToken
execute_approved_query(token: ApprovalToken) → ExecutionResult
summarize_safe_result(result: SafeResultBundle) → FinalAnswer

MVP Nodes:

Diagram node/gate	Concrete implementation
1. UserPrompt	FastAPI receives {session_id, user_id, prompt} . Store raw prompt only if policy allows; otherwise store prompt_hash plus redacted prompt. Create AgentState(run_id, prompt, user_context, budget) .
2. InputPolicyGate	Deterministic first: regex/blocklist for SQL verbs, direct identifiers, MRN/name/DOB/contact patterns, “which patient”/“show me rows” intent. Claude classifier second, returning QuestionIntent.safety_label

	s . Code decides. Reject destructive, raw PHI/PII, patient-identifying, permission-changing, or operational DB admin requests. The brainstorm doc explicitly requires rejecting direct PII/PHI and individual-level requests.
3. IntentClassifier	Claude structured output into <code>QuestionIntent</code> . Categories: <code>schema_discovery</code> , <code>join_help</code> , <code>aggregate_query</code> , <code>cohort_definition</code> , <code>clarification_needed</code> , <code>disallowed</code> . Extract slots: metric, entity, time window, grouping, filters, requested grain. If <code>schema_discovery</code> or <code>join_help</code> , skip BigQuery execution and return catalog/docs only.
4. Clarification	Deterministic routing when required fields are absent. Ask one precise question, not a broad prompt. Examples: missing time range, undefined department, ambiguous metric, unclear encounter type. Re-enter at <code>IntentClassifier</code> ; do not retrieve or query while ambiguous.
5. RetrievalPermissionGate	Compute <code>PermissionScope</code> from user role plus dataset/table/column allowlists. Apply sensitivity tags before retrieval so forbidden columns never enter LLM context. This gate implements the doc's requirement to restrict retrieval to

	approved Epic tables/columns and mask sensitive data.
6. RetrievalLayer	Query Postgres in three ways: lexical FTS for exact table/column terms, vector search for semantic phrases, and graph search over <code>join_edge</code> . Fuse with RRF: $\text{score} = \text{lexical_rrf} + \text{vector_rrf} + \text{exact_identifier_boost} + \text{join_path_boost} - \text{sensitivity_penalty}$. Return bounded context only: top 5 tables, top 30 columns, top 5 join paths, citations. The brainstorm doc correctly says vector search alone is insufficient and calls for lexical + vector + relationship retrieval.
7. RetrievedContextGate	Deterministic checks: at least one table above threshold; requested metric column exists; time column exists if prompt has time window; group-by columns exist; join path exists if >1 table; all candidates are allowed; conflicts/gaps are explicit. If weak context, run one retrieval expansion; then clarify/refuse.
8. QueryPlanAST	Claude creates <code>QueryPlan</code> , not SQL. Inputs are only prompt, intent, permission scope, and bounded retrieved context. The plan must include target metric, tables, columns, filters, joins, aggregations, time constraints, groupings, expected output, and citations—the

	exact layout already sketched in the brainstorm doc.
9. PlanSafetyGate	Deterministic validation of <code>QueryPlan</code> : no new objective beyond prompt; tables/columns subset of permission scope; sensitive fields not projected; identifiers not requested; aggregate-only if user lacks row-level clearance; estimated complexity acceptable; partition/time filters required for large facts.
10. SQLGeneration	Prefer deterministic <code>plan_to_bigquery_sql(plan)</code> . Allow Claude SQL generation only as fallback and only from <code>ApprovedQueryPlan</code> . All user literal values become named BigQuery parameters. Identifiers cannot be parameters, so identifiers must come only from validated catalog refs. No Python code generation in MVP.
11. SQLStaticValidation	SQLGlot parse with BigQuery dialect. Accept only one statement, root <code>SELECT</code> or <code>WITH</code> . Reject DDL/DML/DCL/scripting: <code>DROP</code> , <code>DELETE</code> , <code>UPDATE</code> , <code>MERGE</code> , <code>INSERT</code> , <code>CREATE</code> , <code>ALTER</code> , <code>TRUNCATE</code> , <code>EXPORT DATA</code> , <code>CALL</code> , <code>EXECUTE IMMEDIATE</code> , <code>DECLARE</code> , <code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> . Reject <code>SELECT *</code> , struct star, UDFs, remote functions, external connections, wildcard

	tables unless explicitly approved and suffix-bounded. Validate every table and column against <code>PermissionScope</code> and BigQuery schema. Check SQL AST matches <code>ApprovedQueryPlan</code>. The brainstorm doc explicitly says AST validation with SQLGlot, not regex-only.
12. <code>BigQueryDryRun</code>	Run dry-run only. Capture <code>total_bytes_processed</code>, SQL validity, project/location, validator result, and referenced tables where available. Important correction: dry run estimates bytes, not true runtime; runtime should be enforced later with timeout.
13. <code>CostExecutionGate</code>	Check dry-run bytes against config, reject high-cost scans, enforce partition filters, reject <code>LIMIT</code> as a cost-control substitute because BigQuery cost is based on bytes read and <code>LIMIT</code> may not reduce compute costs. Create an opaque <code>approval_token = HMAC(sql_hash, plan_hash, scope_hash, max_bytes, expiry)</code>. Execution must require this token. (Google Cloud Documentation)
14. <code>ReadOnlyQueryExecution</code>	Execute only via <code>execute_approved_query(approval_token)</code>. Recompute hashes before running. Use read-only service account, <code>maximum_bytes_billed</code>,

	<code>job_timeout_ms</code> , query parameters, labels, fixed location, no destination table, no write disposition, no UDF resources, no external table definitions. Return raw result only to <code>ResultSafetyGate</code> , not directly to the LLM.
15. <code>ResultSafetyGate</code>	Deterministic output policy. Reject/redact if columns include identifiers or denied sensitivity tags. Enforce row cap and configurable <code>min_cell_count</code> for aggregate cells. For MVP, treat patient, encounter, account, MRN, name, DOB, address, phone, email as non-returnable identifiers unless governance explicitly approves a different policy.
16. <code>InterpretationCitations</code>	Claude may summarize only the safe result payload plus SQL, result shape, caveats, and citations. Always show assumptions, table/column citations, and limitations. The brainstorm doc calls out misleading interpretation as a risk and recommends showing SQL, result shape, assumptions, caveats, and source citations.
17. <code>BoundedFollowUp</code>	Classify follow-up as refinement, new request, or unsafe. Reuse prior approved context only when scope does not broaden. No new permissions, no new tools, no agent spawning. Suggested caps: retrieval expansions ≤ 2, SQL repairs ≤ 2, cost

replans ≤ 1 , full graph recursion limit ≤ 25 . The brainstorm doc's follow-up controller already requires max retry limits and re-entry from the latest correct node.

MVP Plan:

Track 1 — Context, retrieval, and planning

Owned nodes: `InputPolicyGate`, `IntentClassifier`, `Clarification`, `RetrievalPermissionGate`, `RetrievalLayer`, `RetrievedContextGate`, `QueryPlanAST`, `PlanSafetyGate`.

Build items:

1. Pydantic models through `ApprovedQueryPlan`.
2. Epic HTML ingestion pipeline with BeautifulSoup/lxml.
3. Postgres catalog schema: `epic_doc`, `table_card`, `column_card`, `join_edge`, `semantic_term`.
4. Hybrid retrieval: Postgres FTS + pgvector + RRF + join graph traversal.
5. Permission masking before retrieval.
6. Context gate thresholds and clarification routing.
7. Claude structured-output prompts for intent and query plan.
8. Retrieval eval set with recall@k, MRR, and “context sufficient?” labels.

Definition of done:

- Given 30–50 gold questions, retrieval returns expected table in top 5 and expected columns in top 20.
- No forbidden/sensitive columns enter `RetrievedContext`.
- Ambiguous prompts route to clarification.
- Output to Track 2 is a valid `ApprovedQueryPlan` or a structured refusal/clarification.

Track 2 — SQL safety, BigQuery execution, results, and app shell

Owned nodes: `SQLGeneration`, `SQLStaticValidation`, `BigQueryDryRun`, `CostExecutionGate`, `ReadOnlyQueryExecution`, `ResultSafetyGate`, `InterpretationCitations`, `BoundedFollowUp`, plus FastAPI/Streamlit/audit.

Build items:

1. LangGraph skeleton and persisted AgentState .
2. Deterministic plan_to_bigquery_sql .
3. SQLGlot BigQuery validator.
4. BigQuery safe client wrapper with dry run, maximum_bytes_billed , timeout, labels, and parameter binding.
5. Approval-token execution barrier.
6. Result safety/redaction gate.
7. Final answer formatter with SQL, result shape, assumptions, citations, and limitations.
8. FastAPI endpoints and Streamlit demo.
9. Audit logging and trace viewer.
10. Gate/unit tests with malicious SQL and unsafe prompt fixtures.

Definition of done:

- No query can execute without a matching approval token.
- Static validator blocks all prohibited SQL classes from the brainstorm doc.
- Dry-run bytes and max-billed cap are logged before execution.
- Result gate blocks identifier columns and unsafe row-level outputs.
- Streamlit demo shows trace, SQL, bytes scanned, citations, and refusal reasons.

NODE & DOCUMENT REFERENCES

Node	Document	Notes	Rough Code Plan
Overall Architecture	Building Effective AI Agents Graph API overview - Docs by LangChain	• MVP: controlled workflow, prompt chaining • implement augmentations with model context protocol • chain of LLM calls, where each one builds off the previous • programmatic checks ('gates') ensure workflow is on track	• graph class StateGraph parameterized by State object • define State: schema of the graph (input schema to all Nodes and Edges, TypedDict or Pydantic) and reducer functions (tell you how to

		• make each LLM call and easier task • LangGraph makes agent workflows like graphs ◦ state: Data structure that represents current state of application ◦ nodes: functions that encode logic of agents. receive state as input, and return an updated state ◦ edge: functions that determine which node to execute based on the current state (conditional branch or fixed transition)	apply updates to the state) ◦ Schema: use TypedDict, dataclass for default values, or use Pydantic BaseModel for recursive data validation • compile with .compile(...)
UserQuestion & Policy Gate	🔗 Prebuilt middlewares - Docs by LangChain 🔗 Hooks reference - Claude Code Docs	• ways to intercept inputs before model/tool: • middleware	
Input / Policy Gate	🔗 Prebuilt middlewares - Docs by L	• tool call limits • PII check	

	angChai n Hooks referenc e - Claud e Code D ocs	early termination/permission check	
Intent Classifier	Model s	QuestionIntent Metric TimeWindow ClarificationNeeded	LLM node to classify the intent of a user query
Clarification Node	Graph API over view - D ocs by L angChai n	how to route to clarification if intent is incomplete, then loop back to intent classifier:	
Retrieval Permission Gate	Resour ces - Mo del Cont ext Proto col Config ure perm issions - Claude C ode Doc s	- how to expose only allowed docs/schema/tools:	
Retrieval Layer	Beautiful Soup Docume ntation — Beauti ful Soup	- Parse HTML into cards and citation chunks - Lexical search vs vector search vs graph retrieval	

	4.14.3 documentation Vector DB		
Semantic Layer	NL2SQL Paper	Map concepts to table definitions	
Retrieved Context Gate		How to add relevance score, required columns, join path, citations:	RetrievedContext and ContextGateDecision
Query Plan AST			
Plan Safety Gate	sqlglot API documentation		
SQL/Python Generation		- how to generate SQL from typed plan	MVP: SQL
BigQuery Dry Run	Dry run query BigQuery Google Cloud Documentation	how to estimate bytes:	
Cost + Execution gate	Estimate and control costs BigQuery Google Cloud		

	oud Doc umentati on		
Read Only Execution	 Python Client for Google B igQuery Pytho n client li braries Google C loud Doc umentati on	• make read only • capture job ID • no secrets exposed	
Result Safety Gate	Middlewa re/Guardr ails		
Intepreta tion + Citations			
Audit Log	FastAPI	• take notes on log prompt hash, retrieved doc ID, SQL hash, dry-run bytes, validation decisions, BigQuery job ID,final answer, failure reason	